

Weekly Progress Report - Room 315 VLA Project

Focus: Adding PDDL/PlanSys2 for Expert Scenario Generation

1. Main Idea

During the past week, my work focused on improving the scenario-generation pipeline of the Room 315 VLA project by introducing PDDL/PlanSys2 as an expert planning layer. The goal is to generate structured and reliable task demonstrations that can later be used to train the VLA model.

2. Why PDDL/PlanSys2

PDDL/PlanSys2 allows the task to be described at a symbolic level. Instead of manually scripting every step required to move a shuttle from Yaskawa to Staubli, the system defines the goal and lets PlanSys2 generate a logical plan, such as preparing switches, opening stoppers, moving the shuttle, stopping it at the target station, and finishing the task.

3. Symbolic Plan vs. Primitive Command

A symbolic plan describes what should happen at a high level, while primitive commands describe how the task is executed in the simulator. For example, a symbolic action such as `move_shuttle` is translated into a concrete VLA command or action vector that can be sent to the VLA Supervisor.

4. Safety and Validation

The VLA Supervisor remains the safety layer. It receives commands from expert tools or from the learned model, checks whether they are valid and safe, and only then allows execution. This prevents unsafe actions such as changing a switch near an occupied protected segment or sending an ambiguous shuttle command when multiple shuttles are present.

5. Dataset Reliability

To avoid training the model on incorrect scenarios, a generated scenario should not be approved for training unless it passes all checks: the symbolic plan is valid, the translated commands are correct, the Supervisor accepts the commands, the shuttle reaches the expected target, and the final goal is truly satisfied.

Generate	Validate	Execute	Verify	Approve for Training
----------	----------	---------	--------	----------------------

6. Dataset Structure

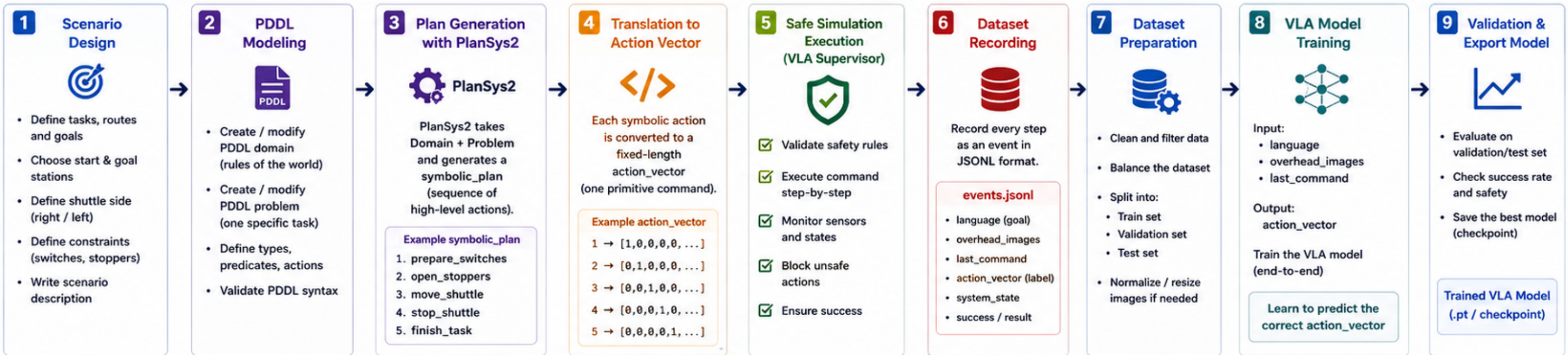
Each episode contains an `events.jsonl` file with decision events for that scenario. Approved events from multiple episodes can later be extracted into `training_events.jsonl`, which is more suitable for model training. The model should not be trained on `data.jsonl` because it mainly contains raw replay and debug data.

Component	Role
PDDL/PlanSys2	Generates high-level expert symbolic plans.
Plan Translator	Converts symbolic steps into primitive VLA commands/action vectors.
VLA Supervisor	Validates safety and executes accepted commands only.
Dataset Recorder	Records decision events for each episode.
Event Extractor	Builds <code>training_events.jsonl</code> from approved episodes.

Next step: implement the validation gate strictly, save validation metadata per episode, and ensure that only successful and verified scenarios are included in the final training dataset.

1 DURING TRAINING: EXPERT DATA GENERATION + VLA MODEL TRAINING

We use PDDL/PlanSys2 as an expert to generate correct action sequences, record data in simulation, and train the VLA model.



KEY IDEA

PDDL/PlanSys2 acts as an expert teacher to generate correct action sequences. These are converted to action_vector labels and used to train the VLA model.



expert knowledge (PDDL)



correct plans (symbolic level)



action_vector (primitives)



dataset (events.jsonl)



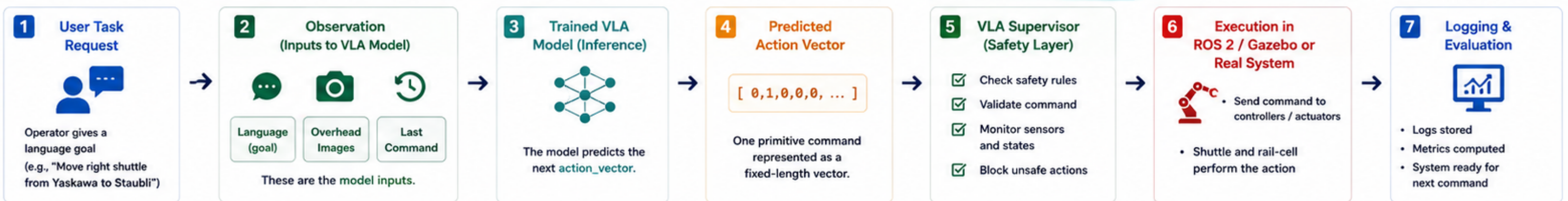
train VLA model



trained model (checkpoint)

2 DURING FINAL OPERATION AFTER SYSTEM DEPLOYMENT

After training, the deployed VLA model predicts action_vector directly from language + observations. PlanSys2 is **NOT** used here.



In final operation: PDDL/PlanSys2 is not in the loop.



The trained VLA model makes decisions based on observations,



while the VLA Supervisor ensures safe and reliable execution.

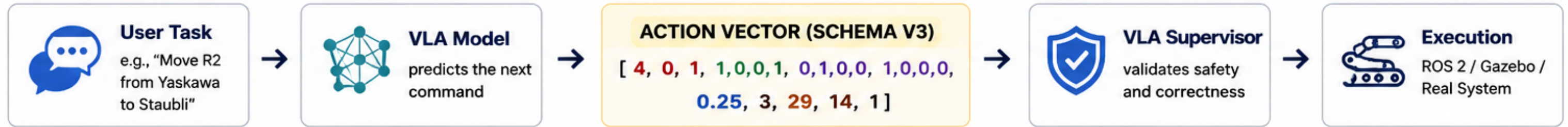


Safe, Efficient, Autonomous

Figure 1 - PDDL/PlanSys2 expert generation during training and VLA inference after deployment

CURRENT ACTION VECTOR (SCHEMA V3)

Action Vector is the numeric, fixed-length command representation used by the VLA Model and the VLA Supervisor.
It encodes **WHAT** to do, **ON WHICH SIDE**, **FOR WHICH SHUTTLE**, and **HOW** to do it safely.



ACTION VECTOR FIELDS (SCHEMA V3)

Index	Field	Size	Type	Description	Example
0	primitive_id	1	int	The high-level action type (what to do)	4
1	side_id	1	int	0 = right, 1 = left	0
2	shuttle_index	1	int	Target shuttle index on the selected side 0 = first shuttle, 1 = second, ... (max 4)	1
3-6	switch_mask_A1..A4	4	int (0/1)	Which switches (A1..A4) will be modified 1 = apply, 0 = ignore	1, 0, 0, 1
7-10	switch_value_A1..A4	4	int (0/1)	Desired values for switches (A1..A4) 1 = straight, 0 = diverging	0, 1, 0, 0
11-14	stopper_mask_A1..A4	4	int (0/1)	Which stoppers (A1..A4) will be modified 1 = apply, 0 = ignore	1, 0, 0, 0
15-18	stopper_value_A1..A4	4	int (0/1)	Desired values for stoppers (A1..A4) 1 = open, 0 = close	0, 0, 0, 0
19	speed_mps	1	float	Shuttle speed in meters/second	0.25
20	wait_condition_id	1	int	Safety / waiting condition before execution	3
21	target_id	1	int	Target destination or objective	29
22	reason_id	1	int	Reason or context of this decision	14
23	coordination_mode	1	int	How to coordinate with other shuttles	1

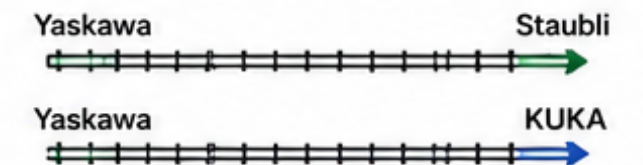
TOTAL LENGTH = 24

PRIMITIVE IDs (primitive_id)

0	WAIT	Wait / hold (no change)
1	DONE	Task finished
2	SET_SWITCHES	Set switches (A1..A4)
3	SET_STOPPERS	Set stoppers (A1..A4)
4	SHUTTLE_ON	Move shuttle (start moving)
5	STOP_NOW	Stop shuttle immediately
6	EMERGENCY_STOP	Emergency stop (all)

SIDE IDs (side_id)

0 = Right side
1 = Left side



SHUTTLE INDEX (shuttle_index)

0 = first shuttle (R1 / L1)
1 = second shuttle (R2 / L2)
2 = third shuttle (R3 / L3)
3 = fourth shuttle (R4 / L4)

Max 4 shuttles
per side

NOTES

- Schema V3 supports up to 4 shuttles per side.
- switch_* and stopper_* masks control which devices are affected.
- wait_condition_id and coordination_mode are used for safe multi-shuttle operations (e.g., block clearance, headway, reservations).
- The vector is decoded by the VLA Supervisor and safety-checked before any execution.

EXAMPLE: SHUTTLE_ON on RIGHT side, targeting RIGHT_SHUTTLE_2

0	1	2	3-6	7-10	11-14	15-18	19	20	21	22	23
4	0	1	1,0,0,1	0,1,0,0	1,0,0,0	0,0,0,0	0.25	3	29	14	1
Action (What/Where/Which)			Switches (A1..A4)		Stoppers (A1..A4)		Speed (m/s)	Wait Cond.	Target ID	Reason ID	Coord. Mode

Figure 2 - Current Action Vector representation, Schema V3

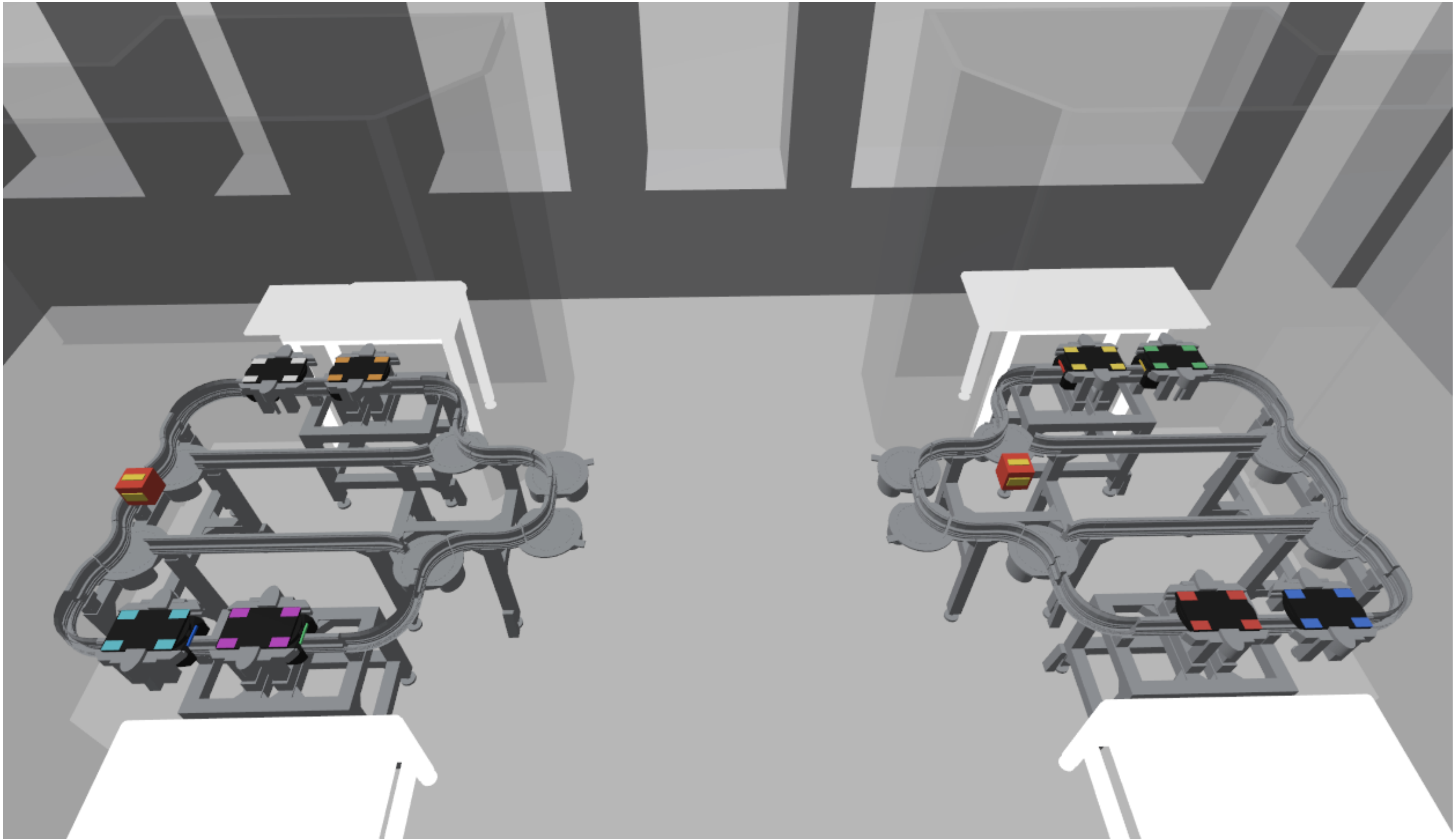


Figure 3 - Room 315 rail-cell simulation view